SIMATS ENGINEERING
ASSESSMENT TOOL 1
COURSE NAME: Artificial Intelligence   COURSE CODE: CSA1709

CO1-AT1-Analytical Problem Solving

Register Number	**192511061**
Name	**Vidhushini.T.S**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. *(BL3)*

Assessment Tool Weightage: 40%

QUESTIONS: 5   **Total Marks: 50**

1. Solve the Water Jug problem: you are given 2 jugs, a 4-gallon one and 3-gallon one. Neither has any measuring maker on it. There is a pump that can be used to fill the jugs with water. How can you get exactly 2 gallons of water into 4-gallon jug? Explicit assumptions: A jug can be filled from the pump, water can be poured out of a jug onto the ground, water can be poured from one jug to another and that there are no other measuring devices available.

2. Find out how Mars Rover, analyse and explore the samples on Mars by answering the following questions.
   i. What are the percept's for this agent?
   ii. Characterize the operating environment.
   iii. What are the actions the agent can take?
   iv. How can one evaluate the performance of the agent?
   v. What sort of agent architecture do you think is most suitable for this agent? Why?

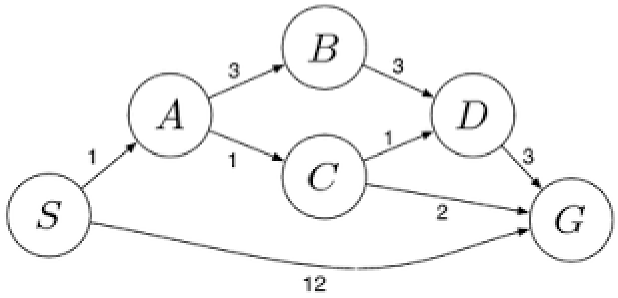
3. Explore the search space using search strategies and formulate the problem components for the 8-Queens problem using the following information: Place 8 queens on a chessboard such that none of the queen's attack any of the others. A configuration of 8 queens on the board as shown in the figure below, but this does not represent a solution as the queen in the first column is on the same diagonal as the queen in the last column.

4. A customer wants to travel from the one location to another location using OLA Cab booking mobile application. The customer can select the any of the cab type as mini, micro, sedan, shared, prime and so on based on his comfort to reach the destination. Identify what type of problem-solving agent can be used and write the pseudocode for the above problem.

5. A logistics company operates a delivery network where packages must be transported between warehouses at minimum cost. Each route between warehouses has an associated travel cost (fuel + time). The company wants to determine the least-cost

path from the starting warehouse **S** to the destination warehouse G using the Uniform Cost Search (UCS) algorithm.

The delivery network is represented as a weighted graph:



Code of Conduct:

I Vidhushini.T.S(192511061) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Vidhushini.T.S

RUBRICS FOR CALCULATION:

Criterion	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Max Marks
Problem Understanding	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem	10
Method Selection	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method	10
Solution Process	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process	12.5
Accuracy of Calculation	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations	10
Interpretation of Results	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation	7.5

1. Water Jug Problem :

Problem :

In this problem, there are two water jugs. One jug has a capacity of 4 gallons and the other has a capacity of 3 gallons. Neither jug has any measuring marks. A water pump is available to fill the jugs, and water can also be poured out onto the ground or transferred from one jug to another. The objective is to obtain exactly 2 gallons of water in the 4-gallon jug.

Problem Formulation:

Initial State: Initially, both jugs are empty.

State: (0,0)

Goal State : The goal is to have exactly 2 gallons of water in the 4-gallon jug.

Goal State: (2,0)

State Space Representation :

Each state is represented as:

(Amount in 4-gallon jug, Amount in 3-gallon jug)

where:

- First value represents the amount of water in the 4-gallon jug.
- Second value represents the amount of water in the 3-gallon jug.

Example:

- (0,0)
- (2,0)

Each state represents the amount of water available in both jugs after performing an action.

Step-by-Step Solution:

Step	State (4G,3G)	Action Performed
1	(0,0)	Both jugs are empty.
2	(0,3)	Fill the 3-gallon jug completely.
3	(3,0)	Pour the 3 gallons into the 4-gallon jug.
4	(3,3)	Fill the 3-gallon jug again.

5	(4,2)	Pour water from the 3-gallon jug into the 4-gallon jug until it becomes full. Now only 2 gallons remain in the 3-gallon jug.
6	(0,2)	Empty the 4-gallon jug onto the ground.
7	(2,0)	Transfer the remaining 2 gallons from the 3-gallon jug into the 4-gallon jug.

The required amount of water has now obtained.

State Space Diagram

```
        (0,0)
          |
    Fill 3-Gallon (0,3)
          |
   Pour into 4-Gallon (3,0)
          |
    Fill 3-Gallon  (3,3)
          |
Pour until 4-Gallon Full (4,2)
          |
  Empty 4-Gallon Jug  (0,2)
          |
Pour into 4-Gallon Jug (2,0)
          |
  Goal Reached
```

Search Strategy :

The search can use Breadth-First Search (BFS) or Depth-First Search (DFS). Since BFS explores states level by level, it is commonly used because it finds the shortest sequence of actions that leads to the solution.

The Water Jug Problem is search problem because:

- The initial state is clearly known.
- The goal state is clearly specified.
- The possible actions are predefined.
- Every action leads to a new state.
- The solution is obtained by searching through the state space.

2. Mars Rover Agent :

Introduction :

A Mars Rover is an intelligent robotic vehicle sent to Mars to explore its surface without human intervention. It is designed to move across different terrains, collect scientific data, capture images, analyze rocks and soil, and send the collected information back to Earth. Since communication between Earth and Mars takes several minutes, the rover must be capable of making many decisions on its own. Therefore, it is considered an intelligent agent in Artificial Intelligence.

i. Percepts of the Mars Rover:

Percepts are the information that an intelligent agent receives from its environment through sensors. The Mars Rover depends on different sensors to understand its surroundings and make decisions.

The main percepts of the Mars Rover are:

- Images captured using navigation and scientific cameras.
- Soil composition detected using onboard scientific instruments.
- Rock properties such as texture, hardness, and mineral content.
- Surface temperature and atmospheric conditions.
- Distance to nearby obstacles using sensors.
- Battery level and available power.
- Current location, direction, and movement status.
- Wheel condition and terrain information.
- Dust level and weather conditions.
- Commands received from scientists on Earth.

These percepts help the rover understand the environment before performing any action.

ii. Operating Environment:

The characteristics of its operating environment are explained below.

Partially Observable: The rover cannot see the entire Martian surface at once. It only receives information about the area around it through its sensors. Therefore, it has incomplete knowledge of the environment.

Dynamic Environment: Although Mars changes slowly, conditions such as dust storms, sunlight, and terrain can change over time. The rover must continuously monitor these changes while performing its tasks.

Sequential Environment: Every action performed by the rover affects its future actions. For example, choosing one path may consume more battery and influence future exploration.

Continuous Environment: The rover moves continuously across the surface. Position, speed, temperature, and battery level change continuously instead of having only fixed values.

Single-Agent Environment: The Mars Rover performs its mission independently. It does not compete or cooperate with other intelligent agents during normal operations.

Known Environment: The rover is programmed with knowledge about its objectives and operating rules before the mission begins. However, it still encounters unknown terrains that require autonomous decision-making.

iii. Actions Performed by the Mars Rover:

Based on the information collected from its sensors, the Mars Rover performs several actions.

Some of the important actions are:

- Move forward, backward, left, or right.
- Stop when an obstacle is detected.
- Capture high-resolution photographs.
- Drill rocks to collect samples.
- Collect soil samples for scientific analysis.
- Analyze rocks and soil using onboard instruments.
- Store collected scientific data.
- Send images and scientific information back to Earth.
- Recharge batteries using solar panels whenever sunlight is available.
- Plan a safer path when obstacles are found.
- Monitor its own health, battery, and wheel condition.

iv. Performance Measure :

The performance measure defines how successfully the Mars Rover completes its mission.

The performance of the rover can be evaluated using the following factors:

Successful Exploration: The rover should explore as much area as possible without getting damaged.

Scientific Discoveries: The number and quality of rock and soil samples collected should be high.

Accurate Data Collection: The images and scientific measurements should be accurate and useful for researchers.

Safe Navigation: The rover should avoid obstacles, steep slopes, and dangerous areas while moving.

Efficient Battery Usage: Power should be used efficiently so that the rover can continue working for a long period.

Reliable Communication: The rover should successfully transmit collected data and images back to Earth.

Mission Completion: The rover should complete all assigned exploration tasks within the available resources.

A rover that safely explores more area while collecting valuable scientific information is considered to have high performance.

v. Suitable Agent Architecture :

The most suitable architecture for the Mars Rover is a Utility-Based Agent.

A Utility-Based Agent evaluates different possible actions and selects the one that provides the highest overall benefit. Instead of simply reaching a goal, it also considers factors such as safety, battery power, travel distance, scientific importance, and environmental conditions.

For example, suppose the rover detects two possible paths:

- One path is shorter but contains many rocks and obstacles.
- The other path is longer but much safer.

A simple goal-based agent may choose the shorter route because it reaches the destination quickly. However, a utility-based agent compares both paths and selects the safer one if it increases the chances of successfully completing the mission.

PEAS Representation of Mars Rover

Component	Description
Performance Measure	Safe exploration, accurate scientific data, efficient battery usage, successful communication
Environment	Mars surface, rocks, soil, atmosphere, craters, dust storms, sunlight
Actuators	Wheels, robotic arm, drilling mechanism, cameras, antenna, solar panels
Sensors	Cameras, temperature sensors, obstacle detectors, chemical sensors, battery sensors, navigation sensors

3. 8-Queens Problem:

Introduction :

The 8-Queens Problem is a classical search problem in Artificial Intelligence. The objective is to place eight queens on an 8 × 8 chessboard so that no queen attacks another queen. Since a queen can move horizontally, vertically, and diagonally, no two queens should be placed in the same row, column, or diagonal. This problem is commonly used to demonstrate search techniques and problem-solving methods in AI.

Problem Formulation:

Problem formulation is the process of defining the search problem using different components.

1. Initial State:

The initial state is an empty 8 × 8 chessboard where no queens have been placed.

The search begins from the empty board.

2. State Space:

The state space consists of all possible arrangements of queens on the chessboard.

Each state represents a different arrangement of queens. Some arrangements are valid, while others contain conflicts where queens attack each other. The search process explores these states until it finds a valid solution. Since there are many possible arrangements, the state space is very large.

3. Operators :

The operators are the actions that generate new states.

The possible operators are:

- Place a queen in an empty column.
- Move a queen to another row if a conflict occurs.
- Remove a queen during backtracking.
- Continue placing queens until all eight queens are safely positioned.

Each operator changes the current state into a new state.

4. Goal State:

The goal state is reached when all eight queens are placed on the chessboard without attacking each other.

The following conditions must be satisfied:

- No two queens should be in the same row.
- No two queens should be in the same column.
- No two queens should be on the same diagonal.

5. Path Cost:

In the 8-Queens Problem, every queen placement has the same cost.

Therefore, every move is considered to have an equal cost.

Search Space Exploration:

The search process begins by placing one queen in the first column.

The algorithm then attempts to place another queen in the next column. Before placing each queen, it checks whether the selected position is safe.

If the position is safe, the queen is placed. If the position is not safe because another queen attacks it, the algorithm tries another row.

If no safe position is available in the current column, the algorithm returns to the previous column, removes the previously placed queen, and tries another position. This process continues until all eight queens are safely placed.

State Space Flow Chart:

```
          Start
            |
Place Queen in Column 1
            |
          Safe Position?
           /      \
          Yes      No
           |        |
          Place     Try Next Row
          Queen
           |
          Column 2
           |
          Safe Position?
           /      \
          Yes      No
           |        |
          Continue
           |
          ...
           |
          Goal State
```

4. OLA Cab Booking Mobile Application :

Introduction:

OLA is an online cab booking application that helps users travel from one place to another. The customer enters the pickup location and destination, after which the application displays different cab options. The customer selects the preferred cab. The system then books the cab and guides the customer until the journey is completed.

Type of Problem-Solving Agent:

The most suitable agent for this application is a Goal-Based Agent.

A Goal-Based Agent makes decisions based on a specific goal. In this case, the goal is to help the customer reach the destination safely and efficiently. The agent collects information such as the pickup location, destination, available cab types, estimated fare, traffic conditions, and driver availability before selecting the best option.

Unlike a simple reflex agent, a goal-based agent considers different possibilities before making a decision. It chooses the option that best satisfies the customer's requirements.

Why Goal-Based Agent is Suitable

A Goal-Based Agent is suitable for the OLA Cab Booking application because:

- The customer's main goal is to reach the destination.
- The system compares different cab types before booking.
- It checks driver availability in real time.
- It estimates the fare and travel time.
- It selects the nearest available driver.
- It provides route guidance during the journey.
- It completes the trip only after the customer reaches the destination.

Thus, the agent continuously works towards achieving the customer's goal.

Problem Formulation :

Initial State:

The customer opens the OLA application and enters the pickup location and destination.

Goal State:

The customer successfully reaches the destination and completes the payment.

State Space :

The state space includes all possible booking situations, such as:

- Different cab types
- Driver availability
- Traffic conditions
- Estimated fare
- Alternative routes
- Ride confirmation

Operators :

The actions performed by the system include:

- Accept pickup location
- Accept destination
- Display available cab types
- Calculate fare
- Search for nearby drivers
- Confirm booking
- Start ride
- Complete journey
- Generate bill

Working of the OLA Cab Booking Agent

The working of the system can be explained as follows:

1. The customer opens the OLA application.
2. The pickup location and destination are entered.
3. The system identifies nearby drivers.
4. Different cab options such as Mini, Micro, Sedan, Prime, and Shared are displayed.
5. The estimated fare and travel time are calculated.
6. The customer selects the preferred cab.
7. The system checks whether the selected cab is available.
8. If available, the nearest driver is assigned.
9. The driver reaches the pickup location.
10. The trip begins and follows the best available route.
11. The customer reaches the destination.
12. The fare is calculated and payment is completed.

Pseudocode :

```
START
```

Input Pickup Location
Input Destination

Search for nearby drivers

Display available cab types
(Mini, Micro, Sedan, Prime, Shared)

Customer selects a cab

Check driver availability

IF driver is available THEN

    Calculate fare
    Estimate travel time

    Confirm booking

    Assign nearest driver

    Driver reaches pickup location

    Start journey

    Navigate to destination

    Complete journey

    Generate bill

    Receive payment

ELSE

    Display "Selected cab is unavailable"

    Suggest another available cab

ENDIF

STOP

Flow Chart:

```
            Start
              |
       Enter Pickup Location
              |
       Enter Destination
              |
       Search Nearby Drivers
              |
       Display Available Cabs
              |
       Customer Selects Cab
              |
       Driver Available?
          /       \
        Yes        No
         |          |
      Calculate Fare   Show Another Cab
         |
      Confirm Booking
         |
      Assign Driver
         |
      Start Journey
         |
      Reach Destination
         |
      Generate Bill
         |
        Stop
```

Advantages:

Artificial Intelligence improves the efficiency of the cab booking system in many ways.

- It finds the nearest available driver.
- It calculates the fastest route.
- It estimates the travel time accurately.
- It predicts traffic conditions.
- It suggests suitable cab types.
- It reduces waiting time.
- It improves customer satisfaction.

Real-Life Example :

Suppose a customer wants to travel from Home to College.

The customer enters both locations in the OLA application. The system checks nearby drivers and displays different cab options with their estimated fares.

The customer selects the Mini cab because it provides a good balance between cost and comfort. The system confirms the booking, assigns the nearest driver, and the customer reaches the destination successfully.

5. Uniform Cost Search (UCS) :

Introduction :

Uniform Cost Search (UCS) is an uninformed search algorithm used to find the minimum-cost path between a start node and a goal node. UCS considers the cost of each path instead of the number of steps. It always expands the node with the lowest cumulative path cost first. UCS guarantees the optimal solution when all edge costs are positive.

Given Graph

The weighted graph contains the following paths:

- S → A = 1
- S → G = 12
- A → B = 3
- A → C = 1
- B → D = 3
- C → D = 1
- C → G = 2
- D → G = 3

Problem Formulation :

Initial State: The search starts from warehouse S.

Goal State: Reach warehouse G with the minimum travel cost.

State Space: All possible paths from S to G.

Operators: The possible actions are moving from one warehouse to another through the available routes.

Path Cost: The total cost is the sum of all edge costs from the start node to the goal node.

Working of Uniform Cost Search

- Uniform Cost Search always selects the node having the lowest cumulative cost from the priority queue.
- The algorithm continues expanding nodes until the goal node is removed from the priority queue.

Step-by-Step Execution:

Step 1: Start from node S.

Priority Queue:

Node	Cost
S	0

Expand S.

New nodes generated:

- A (Cost = 1)
- G (Cost = 12)

Priority Queue:

Node	Cost
A	1
G	12

Step 2: Expand A because it has the lowest cost.

Generate:

- B = 1 + 3 = 4
- C = 1 + 1 = 2

Priority Queue:

Node	Cost
C	2
B	4
G	12

Step 3: Expand C.

Generate:

- D = 2 + 1 = 3
- G = 2 + 2 = 4

The new path to G (cost = 4) is better than the existing path (cost = 12), so update it.

Priority Queue:

Node	Cost
D	3
B	4
G	4

Step 4: Expand D.

Generate:

- G = 3 + 3 = 6

Since G already has a lower cost (4), ignore this path.

Priority Queue:

Node	Cost
B	4
G	4

Step 5: Expand B.

Generate:

- D = 4 + 3 = 7

Since D has already been reached with cost 3, ignore this path.

Priority Queue:

Node	Cost
G	4

Step 6: Expand G.

The goal node is reached.

The search stops.

Possible Paths:

Path	Total Cost
S → G	12
S → A → B → D → G	1 + 3 + 3 + 3 = 10
S → A → C → D → G	1 + 1 + 1 + 3 = 6
S → A → C → G	1 + 1 + 2 = 4

Least-Cost Path

The minimum-cost path found by Uniform Cost Search is:

S → A → C → G

Total Cost = 1 + 1 + 2 = 4

Pseudocode:

```
UniformCostSearch(Start, Goal)

Create a Priority Queue

Insert Start with cost 0

WHILE Priority Queue is not empty

   Remove node with minimum cost

   IF node = Goal
      Return Solution
```

```
        Expand the node

        FOR each neighboring node

            Calculate cumulative cost

            IF new cost is smaller
                Update Priority Queue

    END WHILE

    Return Failure
```

Advantages of Uniform Cost Search

- Finds the optimal (least-cost) solution.
- Considers path cost instead of only the number of steps.
- Works well for weighted graphs.
- Guarantees the shortest path when all edge costs are positive.
- Widely used in route planning and navigation systems.

Applications:

- GPS navigation systems
- Logistics and delivery route optimization
- Robot path planning
- Network routing
- Airline route planning
- Supply chain management